Round 1 Assignment: SDE Intern (Flutter)

Issued 11 October 2026. Submission deadline: Tuesday, 13 October 2026, 11:59 pm IST.

1. About this assignment

Thank you for your interest in joining Vetifi. This assignment is the first round of our selection process for the founding Flutter engineering team.

Vetifi is a clinical reference app used by veterinarians, often in clinics with poor or no connectivity. The assignment below is a reduced version of problems we work on: drug dose calculations, offline data, and keeping that data correct and safe as it changes.

On the use of AI. We want you to use AI tools. We use them every day ourselves. But we are not hiring you for your prompts; we are hiring you for your judgement. Several parts of this assignment contain problems that AI tools commonly get wrong, and much of the marking depends on whether you noticed them. You should be able to explain every line you submit and why it is there. Use AI, but use your brain as well.

The assignment is deliberately larger than most people can finish in the time given. If you run short of time, work in this order: Part A, Part D, Part E, Part F, Part G, then Parts B and C. State clearly in your README what you completed and what you left out.

The deadline is **Tuesday, 13 October 2026, 11:59 pm IST**. Submissions received earlier will receive bonus credit, provided the quality is maintained. Early submission does not compensate for an incorrect dose calculation.

2. Logistics

Deadline	Tuesday, 13 October 2026, 11:59 pm IST
Submission	Reply to the assignment email with links to (1) a GitHub repository, (2) an Android APK, and (3) a video of up to 5 minutes
Repository	Folders `/app` (Flutter) and `/server` (backend), plus README.md, DECISIONS.md, BREAK_IT.md and AI_LOG.md at the root
Technology	Flutter (stable) for the app. Any language and framework for the backend, with a real database. Firebase, Supabase, Appwrite, PocketBase and similar backend services are not permitted.
Role	Founding team Flutter intern. 4 months, unpaid, remote. Conversion to a full time role is based on performance.

3. Background and data

A veterinarian in a rural clinic has no mobile signal and needs to know how much of a drug to draw up, or what to add to a fluid bag. You are building the tool they use.

Our content team has exported the base dataset below. It came from a real pipeline and has not been cleaned, so please treat it with care. All drug names and doses are fictional and must not be used clinically.


```
{
  "schema_version": 2,
  "dataset_version": "v14",
  "drugs": [
    {"id": "d001", "name": "Veloxamine", "concentration": "50 mg/mL", "route": "IV",
     "doses": {"dog": {"min": 5, "max": 10, "unit": "mg/kg"},
               "cat": {"min": 2, "max": 4, "unit": "mg/kg"}}},
    {"id": "d002", "name": "Tranzorel", "concentration": "1 mg/mL", "route": "IM",
     "doses": {"dog": {"min": 10, "max": 20, "unit": "mcg/kg"},
               "cat": {"min": 5, "max": 10, "unit": "mcg/kg"}}},
    {"id": "d003", "name": "Pirovane", "concentration": "25 mg/mL", "route": "PO",
     "doses": {"dog": {"min": 1, "max": 2, "unit": "mg/kg"}, "cat": null}},
    {"id": "d004", "name": "Meloquin", "concentration": "1.5 mg/mL", "route": "PO",
     "doses": {"dog": {"min": 0.1, "max": 0.2, "unit": "mg/kg"}}},
    {"id": "d005", "name": "Meloquin ", "concentration": "5 mg/mL", "route": "SC",
     "doses": {"dog": {"min": 0.1, "max": 0.2, "unit": "mg/kg"},
               "cat": {"min": 0.05, "max": 0.1, "unit": "mg/kg"}}},
    {"id": "d006", "name": "Corvadil", "concentration": "10 mg/mL", "route": "IV",
     "doses": {"dog": {"min": 8, "max": 4, "unit": "mg/kg"}}},
    {"id": "d007", "name": "Benzaprol", "concentration": "10%", "route": "IV",
     "doses": {"dog": {"min": 2, "max": 5, "unit": "mg/kg"},
               "cat": {"min": 2, "max": 5, "unit": "mg/kg"}}},
    {"id": "d008", "name": "Zentrafen", "concentration": "100 mg/mL", "route": "PO",
     "max_total_mg": 400,
     "doses": {"dog": {"min": 10, "max": 15, "unit": "mg/kg"}}},
    {"id": "d009", "name": "Quilomid", "concentration": "0 mg/mL", "route": "IV",
     "doses": {"dog": {"min": 1, "max": 3, "unit": "mg/kg"}}},
    {"id": "d010", "name": "Dravexin", "concentration": "20 mg/mL", "route": "IM",
     "doses": {"Dog": {"min": 2, "max": 4, "unit": "mg/kg"},
               "feline": {"min": 1, "max": 2, "unit": "mg/kg"}}}
  ]
}
```

Your app will also be tested against other datasets with the same structure, so please do not write fixes for individual entries.

4. Part A: Dose calculator

The user searches for a drug, selects a species, enters body weight, and sees the dose range in mg and the volume range in mL.

1. The app must work offline from first launch, with the base dataset included in the app.
2. Where the data for a drug or species is missing, inconsistent or unusable, the app must say so clearly and must not calculate. Deciding what counts as unusable is part of the task.
3. Weight may be entered in kg or lb, with decimals, and sometimes with a comma as the decimal separator (for example `4,5`). Impossible values must be rejected.
4. The interface should be simple and professional. The final volume must be impossible to misread, and the mg dose and concentration should appear alongside it.

5. Part B: Search

1. Add at least 50,000 generated drug entries to the base dataset.
2. Search must tolerate spelling mistakes; for example, `melokin` and `benzaprl` should find the correct drugs. Exact matches should rank first, then prefix matches, then approximate matches.
3. Results from an earlier, slower query must never replace results from a later one.
4. Report the typical and worst time from keystroke to results on a real midrange Android phone, and explain how you measured it. Typing should not cause dropped frames.

6. Part C: Constant rate infusion

Some drugs are given continuously by adding them to a fluid bag. The update file in Part D introduces one such drug, Dopraxine, with a dose expressed in mcg/kg/min.

The user enters body weight, dose rate, fluid rate (mL/hr) and bag size (mL). The app shows how many mL of drug to add to the bag and how long the bag will last. You will need to work out the formula yourself. Pay close attention to units.

7. Part D: Dataset server

Corrections to drug data must reach users without a new app release. Build a backend service that publishes and serves dataset versions. PostgreSQL is preferred; SQLite is acceptable. Running `docker compose up` should be the only setup step.

The service should provide the following endpoints:

Endpoint	Description
`GET /v1/manifest?schema=2&current=v14`	Returns the latest version the client can use, whether a delta from the client's current version is available, and the payload size and SHA256 hash. Supports ETag, returning 304 when nothing has changed.
`GET /v1/datasets/{version}?from={base}`	Returns a delta from the base version where possible, otherwise a full snapshot. Deltas spanning several versions must be combined correctly.
`POST /v1/admin/datasets`	Publishes a new version. Requires admin authentication. Validates every entry and returns errors that identify the drug and field. Published versions cannot be modified.
`POST /v1/admin/withdraw`	Withdraws a drug, with a reason, as a new version.
`POST /v1/calc_logs`	Accepts calculation records uploaded by the app. A repeated upload must not create duplicates.
`GET /health`	Reports service and database status.

In addition:

1. Every manifest and payload must be signed (Ed25519 is recommended). The app holds only the public key and must reject anything that fails verification. Someone who gains control of the server host or the network path must not be able to change a dose.
2. If two administrators publish from the same base version at the same time, exactly one should succeed and the other should receive 409 Conflict. Repeating a publish request with the same idempotency key should return the original result.
3. The server must reject the same invalid data that the app rejects. Explain how you prevent the two sets of rules from drifting apart.
4. When started with `CHAOS=1`, the server should randomly return a tampered payload, an incorrect checksum, a truncated response or a server error. The app must handle all four without changing its data.
5. Include integration tests that run against a real database, covering at least the concurrent publish case and delta combination.

Seed the server with v14 as the base version. Use the following files as publishing test cases:

```
{
  "schema_version": 2,
  "dataset_version": "v15",
```

```
  "base_version": "v14",
  "withdrawn": ["d009"],
  "upserts": [
    {"id": "d005", "name": "Meloquin Injectable", "concentration": "5 mg/mL", "route": "SC",
     "doses": {"dog": {"min": 0.1, "max": 0.2, "unit": "mg/kg"},
               "cat": {"min": 0.05, "max": 0.1, "unit": "mg/kg"}}},
    {"id": "d007", "name": "Benzaprol", "concentration": "100 mg/mL", "route": "IV",
     "doses": {"dog": {"min": 2, "max": 5, "unit": "mg/kg"},
               "cat": {"min": 2, "max": 5, "unit": "mg/kg"}}},
    {"id": "d011", "name": "Dopraxine", "concentration": "40 mg/mL", "route": "IV CRI",
     "cri": {"dog": {"min": 5, "max": 10, "unit": "mcg/kg/min"}, "cat": null}}
  ]
}
```

File	Contents
`update_v15.json`	As shown above. This should be accepted.
`update_v16.json`	Same as v15, but with `"dataset_version": "v13"`
`update_v17.json`	Same as v15, but with `"schema_version": 3` and `"dataset_version": "v17"`
`update_v18.json`	The v15 file cut off halfway, so it is not valid JSON

Deployment is optional. A working public URL earns bonus credit, but a clean local setup is enough for full marks.

8. Part E: Updates in the app

1. When a connection is available, the app checks the manifest in the background, downloads any update, verifies the signature and hash, and applies it.
2. Updates must be applied atomically. If the app is closed or killed at any point during an update, it must start next time with either the complete old data or the complete new data. Please demonstrate this with a test or a written manual procedure.
3. The dataset version must be visible on every result screen. Withdrawn drugs must not be calculable, and the user should be told why.
4. Each calculation should be stored on the device with its inputs, dataset version and result, and uploaded to `/v1/calc_logs` when a connection is available.
5. No network request may be made at the time of calculation. The app must continue to work offline indefinitely with the data it holds.

9. Part F: Written questions

Please answer the following in DECISIONS.md, in no more than 200 words each. Give a clear answer in each case. We will check that your app and server behave as your answers describe.

1. **Small doses.** A 2.5 kg kitten needs 0.07 mL. A 40 kg dog needs 38.4 mL. What exactly does your app display in each case, and when should it refuse to round?
2. **Missing species data.** A senior vet on the content team suggests: "If the cat dose is missing, show the dog dose with a warning. Vets know what they are doing." Give the strongest argument for this view, then make your decision.
3. **Bag volume.** Adding a drug to a fluid bag increases the volume of the bag. Does this affect your infusion result? Show with numbers when it matters and when it does not, and explain what your app does about it.
4. **Users who never update.** A vet has been offline for five months and still has v14, with the incorrect Benzaprol concentration. What, if anything, can the app do for them?
5. **Leaked signing key.** Your private signing key is published online. What can an attacker do, what happens to installed apps, and how do you recover? If your current design cannot recover without an app release, say so and describe what you would change.
6. **Limits of your design.** Describe one specific situation in which your storage, search, server or update design would be the wrong choice, and what you would use instead.

10. Part G: Code review

The three functions below come from an earlier version of a hypothetical dose app. For each one, list the problems you find, rank them by the harm they could cause to an animal, and write a corrected version of the most serious one. Include your answers in DECISIONS.md.

Function 1 (Dart)

```
double volumeMl(double weight, String unit, double dosePerKg,
    String concentration) {
  final kg = unit == 'lb' ? weight / 2.2 : weight;
  final mgPerMl = double.parse(concentration.split(' ')[0]);
  final doseMg = kg * dosePerKg;
  return double.parse((doseMg / mgPerMl).toStringAsFixed(1));
}
```

Function 2 (Dart)

```
Future<void> applyUpdate(File file) async {
  final db = await openDb();
  await db.delete('drugs');
  final data = jsonDecode(await file.readAsString());
  for (final d in data['upserts']) {
    await db.insert('drugs', Drug.fromJson(d).toMap());
  }
  await prefs.setString('dataset_version', data['dataset_version']);
}
```

Function 3 (Python, FastAPI)

```
@app.post("/admin/datasets")
def publish(body: dict, x_admin_key: str = Header(None)):
    if x_admin_key != os.environ.get("ADMIN_KEY"):
        raise HTTPException(401)
    latest = db.execute("SELECT max(version) FROM datasets").scalar()
    new_version = f"v{int(latest[1:]) + 1}"
    db.execute(f"INSERT INTO datasets (version, payload) "
               f"VALUES ('{new_version}', '{json.dumps(body)}')")
    return {"version": new_version}
```

Each function has more than one serious problem. Assume that callers pass whatever the user entered.

11. Part H: Documentation and video

1. **README.md:** setup steps, a short description of your architecture, your main technical choices, and what you did not complete.
2. **BREAK_IT.md:** the ways in which your app could still show an incorrect or misleading number, whether you have protected against each one, and which test covers it. Name at least one issue you did not fix and say whether you would release the app anyway.
3. **AI_LOG.md:** which AI tools you used and for what; an honest estimate of how much of the code was AI generated; at least two cases where the AI was wrong or unsafe, with the prompt, the relevant part of the response, and what you did instead; and one decision you changed your mind about.
4. **Video** (up to 5 minutes, unedited, with your face and screen visible): demonstrate the app in airplane mode, then online with the server in chaos mode refusing a tampered update. Finish with the part of your work you are least confident about.

Please commit regularly. Your commit history helps us understand how you worked.

12. Evaluation

Area	Weight
Correctness and safety of every result shown or refused (Parts A and C)	20%
Backend: correctness, signing, concurrency, validation and tests (Part D)	20%
App engineering: search, signed atomic updates, log upload (Parts B and E)	20%
Written questions, and whether the code matches the answers (Part F)	15%
Code review (Part G)	10%
Documentation and video (Part H)	15%
Bonus: early submission or public deployment	up to 5%

If anything in this brief is unclear, please reply to the assignment email and we will respond the same day.

Best regards,
Vanshaj
Cofounder and CTO, Vetifi Technologies Private Limited